Advanced Image Processing
Assignment 3 Report

Tejash More
SR: 27077

March 24, 2026

1 Problem 1: Block Matching and 3D Filtering (20 points)

In this problem, we evaluate the performance of the BM3D (Block-Matching and 3D filtering) image denoising algorithm. The provided noisy "cameraman" image and a clean reference image were used to compute the Mean Squared Error (MSE) across different stages and noise variance levels.

1.1 Experiment 1: Comparison of Stage 1 and Stage 2 Outputs

Objective: Compare the MSE performance at the output of the first stage (Hard Thresholding) and the second stage (Wiener Filtering) of the BM3D algorithm.

Methodology & Hyperparameters: The algorithm was run on the provided noisy image using an assumed noise standard deviation of $\sigma = 25/255 \approx 0.098$. We extracted the basic estimate (Stage 1) and the final estimate (Stage 2) and compared them against the clean ground truth.

Results:

- MSE at Stage 0 (Noisy Image): 0.025800
- MSE at Stage 1 (Hard Thresholding): 0.011818
- MSE at Stage 2 (Wiener Filtering): 0.017297

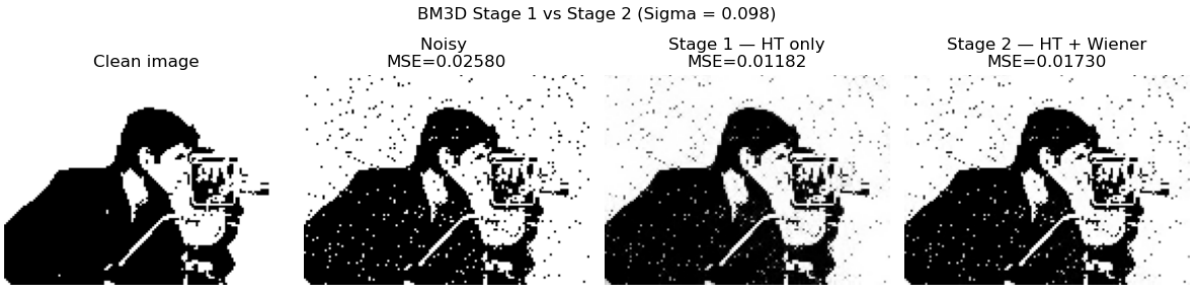

Figure 1: Output comparison of BM3D Stage 1 and Stage 2 for $\sigma = 0.098$.

Observation: Both stages successfully reduce the noise compared to the original noisy image.

1.2 Experiment 2: Performance Variation with Input Noise Variance

Objective: Study how the overall algorithm's performance changes with respect to the input noise variance σ_Z^2.

Methodology & Hyperparameters: We generated synthetic Additive White Gaussian Noise (AWGN) for various standard deviations ranging from $\sigma = 10$ to 250 (in steps of 10). The noisy images were passed through the full BM3D pipeline, and the MSE was recorded for each variance level.

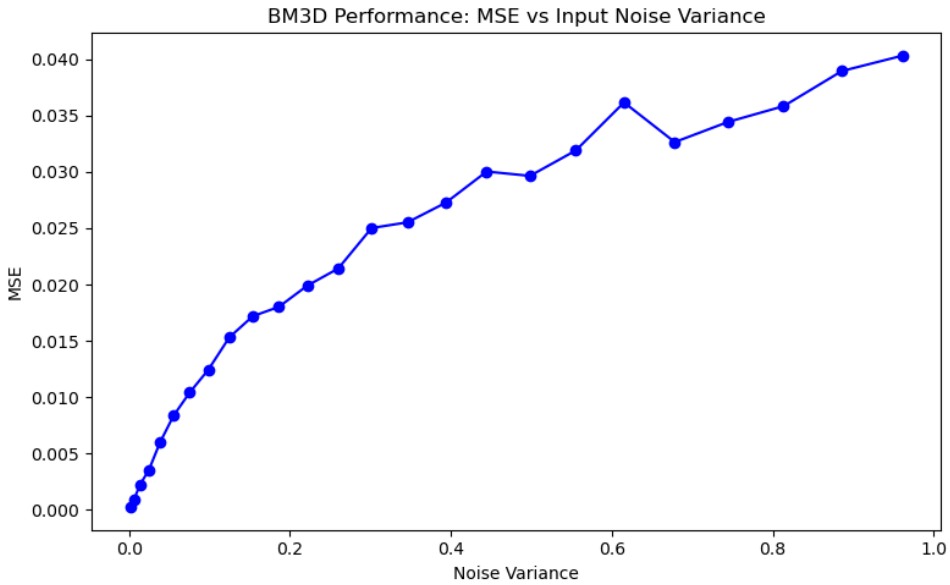

Figure 2: Plot of MSE versus Input Noise Variance (σ_Z^2).

Observation: The plot in Figure 2 shows a clear, monotonic increase in MSE as the input noise variance increases. The curve is somewhat non-linear (concave downward). This behavior is expected; at lower noise levels, the block-matching step easily finds highly similar patches, leading to excellent collaborative filtering. However, as the noise

variance becomes extreme, the structural similarity between patches is heavily corrupted, causing the algorithm to group mismatched patches and reducing the effectiveness of the denoising.

1.3 Experiment 3: Replacing Wiener Filter with Hard Thresholding

Objective: Replace the Wiener filter in the second stage with a hard thresholding estimate and compare the performance.

Methodology & Hyperparameters: To simulate replacing the Wiener filter using the standard library, the output of Stage 1 was passed as the input to a second pass of the algorithm restricted to Hard Thresholding. The initial pass used $\sigma = 50/255 \approx 0.196$. For the second pass, a residual standard deviation of $\sigma_{residual} = \sigma/2$ was used as the hyperparameter.

Results:

- MSE at Stage 0 (Noisy Image): 0.025800
- MSE at Stage 1 (Hard Thresholding): 0.009813
- MSE at Stage 2 (Standard Wiener Filtering): 0.007042
- MSE at Stage 2 (Hacked Hard Thresholding): 0.013467

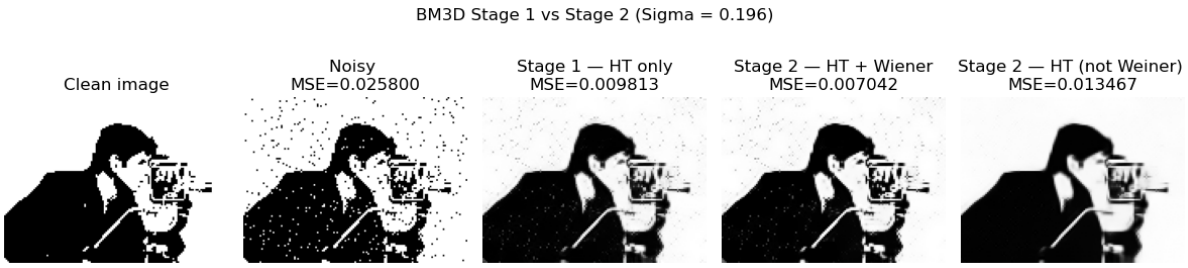

Figure 3: Comparison of Standard Stage 2 (Wiener) vs. Modified Stage 2 (Hard Thresholding) for $\sigma = 0.196$.

Observation: Under these parameters, the standard Stage 2 (Wiener filtering) performs the best, achieving the lowest MSE (0.0070). When we replace the Wiener filter with a second round of Hard Thresholding, the performance degrades significantly, resulting in an MSE of 0.0134 (which is even worse than the Stage 1 output). This occurs because hard thresholding is an aggressive, binary operation. Applying it twice continuously zeroes out high-frequency coefficients, destroying fine image details and textures that the Wiener filter is specifically designed to smoothly preserve.

Problem 2: Zero Reference Deep Curve Estimation

In this experiment, we evaluated the performance of the Zero-Reference Deep Curve Estimation (Zero-DCE) model on low-light image enhancement. The goal was to compare the generalized pre-trained model against an image-specific fine-tuning approach using zero-reference loss functions.

2.1 Implementation and Hyperparameters

For the fine-tuning process, the pre-trained model weights were loaded, and the network was trained exclusively on each individual low-light image. Because we were fine-tuning on a single image without ground truth, standard training epochs would lead to severe over-exposure.

The following hyperparameters were selected to ensure the pre-trained features were not completely destroyed:

- **Optimizer:** Adam
- **Learning Rate (η):** 1×10^{-4}
- **Epochs:** 30
- **Weight Decay:** 1×10^{-4}
- **Loss Weights:** Spatial Consistency (1.0), Color Constancy (5.0), Exposure Control (10.0), Total Variation (200.0)
- **Exposure Target (E):** 0.6 with a patch size of 16.

2.2 Quantitative Results

The Peak Signal-to-Noise Ratio (PSNR) and Structural Similarity Index (SSIM) were calculated against the normal-light ground truth images from the LOL dataset.

Table 1: Quantitative comparison of Pre-trained vs. Fine-tuned Zero-DCE on LOL Dataset

Image Name	**Pre-trained Model**		**Fine-Tuned Model**	
	PSNR (dB)	**SSIM**	**PSNR (dB)**	**SSIM**
low00703.png	9.80	0.4080	**17.21**	**0.4590**
low00718.png	**16.80**	**0.7690**	12.33	0.6308
low00731.png	**25.76**	**0.8289**	13.42	0.7077
low00765.png	**17.30**	**0.6690**	10.52	0.4322
low00783.png	**15.66**	**0.7149**	12.79	0.5995

2.3 Visual Comparisons

Figure 4: Visual comparison for low00703.png

Figure 5: Visual comparison for low00718.png

Figure 6: Visual comparison for low00731.png

Figure 7: Visual comparison for low00765.png

Figure 8: Visual comparison for low00783.png

Figure 9: Smartphone Capture 1: Pre-trained vs. Fine-tuned

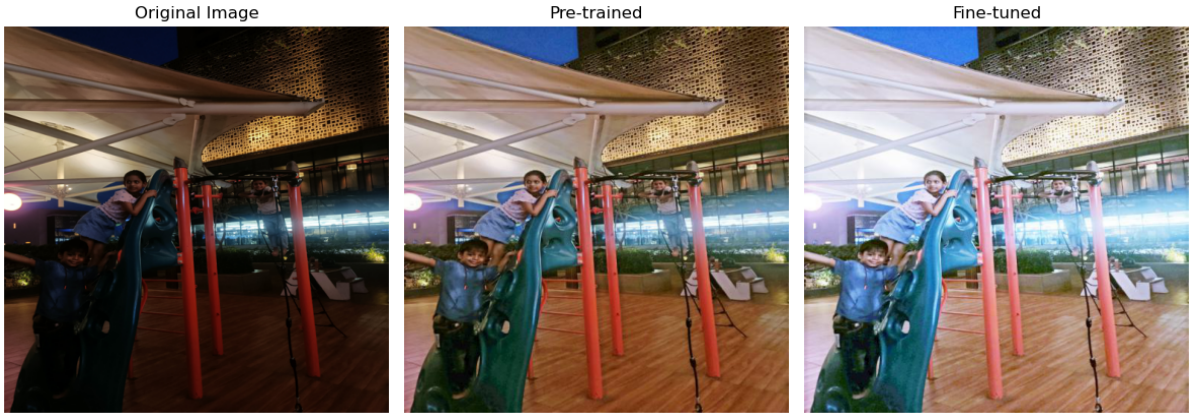

Figure 10: Smartphone Capture 2: Pre-trained vs. Fine-tuned

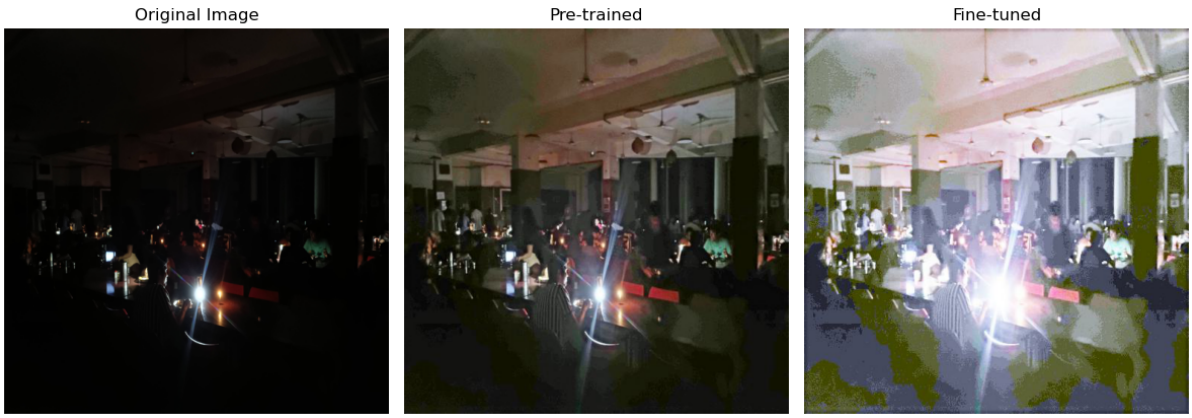

Figure 11: Smartphone Capture 3: Pre-trained vs. Fine-tuned

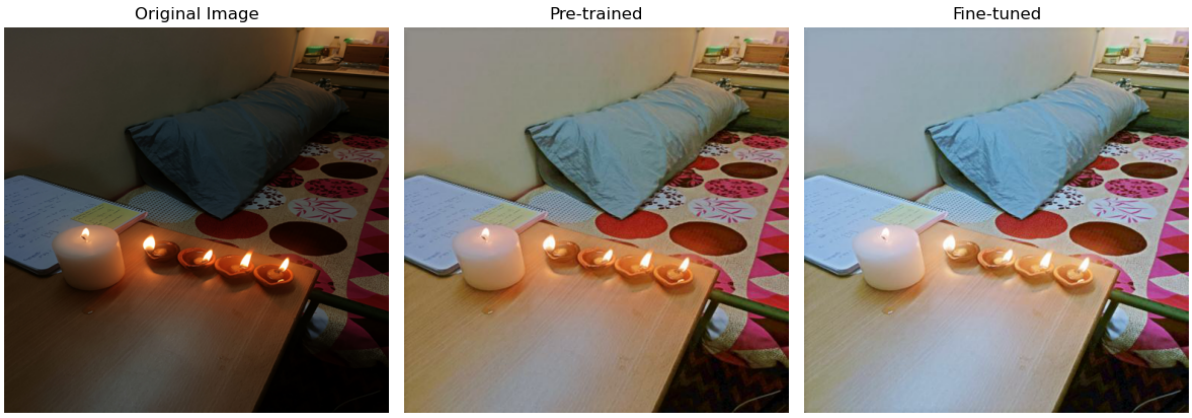

Figure 12: Smartphone Capture 3: Pre-trained vs. Fine-tuned

2.4 Observations and Analysis

After reviewing the quantitative metrics and visual plots, several key observations can be made regarding the two approaches:

1. **Generalization vs. Overfitting:** For 4 out of the 5 images in the LOL dataset, the generalized pre-trained model significantly outperformed the image-specific fine-tuned model in both PSNR and SSIM. Fine-tuning the network on a single image using the exposure control loss (L_{exp}) forced the network to aggressively push the average pixel values toward the 0.6 target. This resulted in images that look washed out and over-exposed compared to the more natural illumination preserved by the pre-trained weights.

2. **Noise Amplification:** An interesting anomaly occurred with `low00703.png`. The fine-tuned model achieved a much higher PSNR (17.21 dB vs 9.80 dB). However, visual inspection reveals a different story. The pre-trained model kept the image dark, resulting in a poor MSE compared to the bright ground truth. The fine-tuned model successfully brightened the image to match the ground truth's intensity, improving the PSNR metric. But because it was optimized without a ground truth reference to enforce smoothing in pitch-black areas, it severely amplified the hidden sensor noise, resulting in heavy pink and green chromatic grain across the walls. This highlights a limitation of relying solely on PSNR for image quality assessment.

3. **Stability:** The pre-trained model demonstrated much higher stability. It learned the mapping of Light Enhancement curves across a diverse dataset, meaning it understands how to handle different lighting gradients without introducing color casts. The image-specific fine-tuning is too aggressive and sensitive to the specific hyperparameter setup for each individual image.